CTF WRITEUP / REVERSE ENGINEERING

activation

FGTE 2025 • Medium • 80 points • Author: aria

Deskripsi soal

Aplikasi meminta username dan sebuah lisensi yang harus cocok dengan identitas tersebut. Tujuannya adalah menelusuri validasi aktivasi untuk mendapatkan secret yang disimpan dalam binary.

Hasil akhir

Username	**RANI**
License	`0905E413DDF6D48B`
Flag	`FGTE{L!c3n$3_4cTiv@t10n_7xY9_b2Q8R5s}`

01 / Identifikasi executable

Perintah `file` menunjukkan program PE32 untuk Windows yang berisi assembly .NET. String UTF-16 juga memperlihatkan pesan validasi dan data Base64 yang relevan.

```
file activator.exe
strings -a -el activator.exe
```

```
$ file activator.exe
activator.exe: PE32 executable for MS Windows 4.00 (console), Intel i386 Mono/.Net assembly, 4 sections

$ strings -a -el activator.exe | grep -E "Activator|Usage|Invalid|Activation|UkFOSQ|c2FsdF9w"
Activator v1.0
Usage: activator.exe <username> <license>
Invalid username.
Invalid license.
Activation OK. Here is your secret:
UkFOSQ==
c2FsdF9w

[+] Metadata .NET, base64 username, dan pesan validasi ditemukan
```

Bukti 01 - screenshot nyata dari terminal: identifikasi .NET dan string yang tersimpan.

02 / Menemukan data tersembunyi

Dari metadata .NET dan CIL ditemukan fungsi `GetEmbeddedUsername`, `GetSalt`, `ExpectedLicense`, `DeriveKeyFromLicense` serta `DecryptBlob`.

Username tersembunyi pada Base64 `UkFOSQ==`, yang didekode menjadi **RANI**. Salt dibangun dari empat bagian Base64, kemudian menghasilkan `salt_part_1_from_cre`.

```
user = base64.b64decode("UkFOSQ==").decode()
salt = base64.b64decode(
    "c2FsdF9wYXJ0XzFfZnJvbV9jcmU="
)
# user = RANI
# salt = salt_part_1_from_cre
```

03 / Menghasilkan lisensi yang benar

Metode `ExpectedLicense` menghitung **HMAC-SHA256(salt, username)**. Program mengambil 8 byte pertama hasil hash dan mengonversinya menjadi hexadecimal uppercase.

```
license = hmac.new(
    salt, user.encode(), hashlib.sha256
).digest()[:8].hex().upper()
print(license)
# 0905E413DDF6D48B
```

```
$ python3 - <<PY
import base64, hmac, hashlib
user = base64.b64decode("UkFOSQ==").decode()
salt = base64.b64decode("c2FsdF9wYXJ0XzFfZnJvbV9jcmU=")
license = hmac.new(salt, user.encode(), hashlib.sha256).digest()[:8].hex().upper()
print("Username:", user)
print("Salt    :", salt.decode())
print("License :", license)
PY
Username: RANI
Salt    : salt_part_1_from_cre
License : 0905E413DDF6D48B
```

Bukti 02 - screenshot nyata: decoding username dan rekonstruksi license.

> **Temuan tambahan:** data mask 8 byte dalam .sdata di-XOR dengan 0x33 dan menghasilkan `fgte_ctf`.

04 / Dekripsi secret menggunakan AES

License valid menjadi bagian dari material key. Fungsi `DeriveKeyFromLicense` menghitung SHA256 dari gabungan string license dan Base64(salt), lalu mengambil 16 byte pertama untuk AES-128.

```
key = hashlib.sha256(
    (license + base64.b64encode(salt).decode()).encode()
).digest()[:16]
# ab5ca2c49a6e6f4425eb73d97f603242
```

Ciphertext berada di `encryptedBlob`, sebuah array 64 byte dari FieldRVA kedua. Enam belas byte pertama adalah **IV**; 48 byte berikutnya merupakan ciphertext AES-CBC dengan PKCS7 padding.

```
iv = encrypted_blob[:16]
ciphertext = encrypted_blob[16:]
dec = Cipher(algorithms.AES(key), modes.CBC(iv)).decryptor()
padded = dec.update(ciphertext) + dec.finalize()
# Lanjutkan dengan PKCS7 unpadding
```

```
$ python3 solve_activation.py activator.exe
[+] Challenge  : activation - FGTE 2025
[+] Binary     : activator.exe
[+] Base64 user: UkFOSQ==
[+] Username   : RANI
[+] Salt       : salt_part_1_from_cre
[+] XOR mask   : fgte_ctf
[+] License    : 0905E413DDF6D48B
[+] AES key    : ab5ca2c49a6e6f4425eb73d97f603242
[+] AES IV     : a7a4b3447a2bdccdddedfc8c5e3ce4a8
[+] Flag       : FGTE{L!c3n$3_4cTiv@t10n_7xY9_b2Q8R5s}

[+] Ciphertext asli diambil langsung dari field RVA binary
[+] Dekripsi AES berhasil dan flag telah diverifikasi
```

Bukti 03 - screenshot nyata dari eksekusi solver Python, bukan flag yang di-hardcode.

> **Flag hasil dekripsi:**
> `FGTE{L!c3n$3_4cTiv@t10n_7xY9_b2Q8R5s}`

05 / Menjalankan program di Windows

Simpan `activator.exe` di folder kerja. Buka PowerShell, lalu masukkan username serta lisensi hasil reverse engineering.

```
PS C:\CTF> .\activator.exe RANI 0905E413DDF6D48B
```

Output yang diharapkan dari alur `Main()` adalah:

```
Activator v1.0
Activation OK. Here is your secret:
FGTE{L!c3n$3_4cTiv@t10n_7xY9_b2Q8R5s}
```

```
$ cat /mnt/data/windows_activation.ps1

# FGTE 2025 - activation
# Simpan activator.exe di folder yang sama.
# Jalankan di PowerShell pada Windows dengan .NET Framework.
cd $PSScriptRoot

Write-Host "=== Identifikasi File ==="
Get-Item .\activator.exe | Select-Object Name, Length

Write-Host "`n=== Eksekusi Activator ==="
.\activator.exe RANI 0905E413DDF6D48B

Write-Host "`nAmbil screenshot PowerShell (Win + Shift + S)."

[CATATAN] Ini screenshot perintah panduan, BUKAN eksekusi Windows.
[CATATAN] Jalankan .ps1 di PowerShell Windows untuk bukti runtime asli.
```

Bukti 04 - screenshot panduan perintah PowerShell; BUKAN screenshot runtime Windows.

Catatan bukti dan reproduksi

Screenshot 01-03 benar-benar diambil dari terminal saat menganalisis file dan menjalankan solver. Lingkungan saat ini tidak memiliki .NET runtime maupun Wine, sehingga .exe belum dijalankan langsung. Untuk mengambil screenshot eksekusi program di Windows, jalankan perintah di atas lalu tekan **Win + Shift + S**.

```
pip install cryptography
python solve_activation.py activator.exe
```

Kesimpulan

Program memakai validasi lisensi lokal yang deterministik dan secret terenkripsi. Dengan membaca Base64 serta metadata .NET, kita dapat merekonstruksi license yang sah, menghasilkan key AES, dan mendekripsi flag secara offline.